

Pay with Ethereum for native LCAI on Lightchain

A user holds **ETH on Ethereum**, swaps it to **LCAI ERC-20** on Uniswap V3, then bridges to **native LCAI on Lightchain (chain 9200)** via the **official** protocol Hyperlane warp — the same route `bridge.lightchain.ai` uses. OrcaVault exposes this as **Pay with Ethereum** so someone can unlock without already holding native LCAI.

Read first. This is **user-signed and non-custodial** — not a hosted custodian, not Circle, not testnet. Native LCAI is **not instant**: after the Ethereum transactions confirm, the official Hyperlane relayer delivers on Lightchain. **LCAI has no Uniswap V2 pool.** Quotes and swaps must use **V3 (0.3% LCAI/WETH)**. A V2 `getAmountsIn` reverts with empty data.

Different from the USDC warp. Base USDC → synthetic USDC on Lightchain is a self-hosted Hyperlane route (you run validator + relayer). This ETH rail uses Lightchain's official LCAI warp. Do not mix the two routers.

Flow — three Ethereum txs, then wait

1. **Quote** — Uniswap V3 Quoter `quoteExactOutputSingle` : how much ETH for N LCAI, plus a buffer for slippage and bridge gas.
2. **Swap** — V3 SwapRouter `exactInputSingle` : ETH → WETH → LCAI ERC-20, fee tier **3000** (0.3%).
3. **Approve** the warp router to spend that LCAI (skip if allowance is already enough).
4. **Bridge** — `transferRemote(9200, recipient, amount)` on the Ethereum collateral warp, paying `quoteGasPayment` in ETH.
5. **Credit** — App verifies the **Ethereum-side** bridge tx (correct warp, dest 9200, recipient, amount). Native LCAI is minted on Lightchain by the official native warp. Do not block the button on L1 inclusion.

Recipient on Lightchain is typically **the app's relay / payment wallet** (address left-padded to bytes32), so the user does not send a second native payment.

Public contracts

Role	Address	Notes
LCAI ERC-20	0x9cA8530CA349c966Fe9ef903Df17a75B8A778927	Official \$LCAI on Ethereum
WETH9	0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2	Canonical
Uniswap V3 SwapRouter	0xE592427A0AEce92De3Edee1F18E0157C05861564	Periphery
Uniswap V3 Quoter	0xb27308f9F90D607463bb33eA1BeBb41C27CE5AB6	ethers v6: <code>.staticCall</code>
LCAI/WETH V3 pool	0x0d047a370611437a1b8e6c2a95ea36f69fdda3be	0.3% fee tier
Warp (Ethereum collateral)	0x01f80bb8e78e79881E8Ec7832fB6C2c59f64e353	<code>transferRemote</code> target
Warp (Lightchain native)	0xEc7096A3116EE769457C939617375Ec1785AA6f1	Mints native LCAI — no ETH-side call
Lightchain domain	9200	Hyperlane dest = chain id

OrcaVault example: native LCAI from this warp is aimed at the published payment / relay wallet on Lightchain (same address as the ERC-8004 card: 2 LCAI or 1 bridged USDC unlock). One Ethereum session can unlock the app.

Implementer notes

- Wallet must be on **Ethereum (chainId 1)** for swap + bridge. Switch to Lightchain **9200** for app use.

- Bridge the **LCAI actually received** after the swap (`balanceOf` before/after), not the quote.
- Example slippage **300 bps** (3%). Example deadline **1200 s**.
- Verify: `tx.to` is the Ethereum warp; selector `transferRemote(uint32,bytes32,uint256)` ; destination **9200**; recipient = your relay; amount $\geq$ fee; plus an ERC-20 `Transfer` of that LCAI from the user **into the warp**.
- Replay-protect Ethereum tx hashes. Kill-switch the rail if the official relayer or the V3 pool is unhealthy.
- Users sign in their own wallet. Never commit keys.

Security checklist

1. Official warp only — do not point `transferRemote` at a random Hyperlane router.
 2. V3 0.3% pool only — V2 will revert.
 3. Credit only after the **bridge** tx confirms on Ethereum, not the swap alone.
 4. Native delivery is the **protocol relayer**, not a Docker agent you run for this rail (unlike the self-hosted USDC warp).
 5. Do not market this as instant or as a CEX on-ramp. It is swap + official bridge.
-

Reference: bridge.lightchain.ai · Uniswap V3 · Hyperlane TokenRouter. Related: Hyperlane USDC warp (self-hosted Base collateral). Public contracts only — no ops keyfiles or test wallets in this document.